

Ramify Vision & Roadmap

This document outlines the strategic vision and feature roadmap for **ramify**. It serves as the master specification for breaking down tasks into **GitHub Issues**.

Project Vision

"Give AI Agents a Safe Space to Experiment and Fail."

ramify aims to become the standard execution & state-management engine for AI Agent frameworks (e.g., Claude Code, LangChain, AutoGen, Custom Agents). Instead of heavy Docker-container-per-task models or risky unisolated local shell execution, **ramify** provides **lightweight, 0.1-second session branching and deterministic rollbacks** using native Git Worktrees and environment isolation.

Roadmap Overview

- ❑ **Phase 1: MVP Core (Stateful Session & Git Isolation)** — *Foundation*
 - ❑ **Phase 2: Agent Usability (Token Optimization & Safety Guard)** — *Agent DX*
 - ❑ **Phase 3: Ecosystem Expansion (Docker Isolation & MCP Server)** — *Integration*
-

Phase 1: MVP Core (Stateful Session & Git Isolation)

Goal: Enable stateful command execution and 100% clean environment rollbacks via Git Worktree.

◆ Issue 1.1: Core Stateful Session (**ramify.Session**)

- **Description:** Implement a **Session** class that tracks current working directory (**cwd**) and environment variables (**env**) across sequential **run()** execution calls.
- **Tasks:**
 - ❑ Implement **Session.run(cmd_str: str) -> CommandResult** using standard **subprocess**.
 - ❑ Track directory changes (e.g., **cd** execution) and persist path state.
 - ❑ Implement environment variable mutation tracking (e.g., **export KEY=VAL**).
- **Acceptance Criteria:** **session.run("cd src"); session.run("pwd")** returns **/path/to/src**.

◆ Issue 1.2: Git-Worktree Branching (**ramify.SessionBranch**)

- **Description:** Implement lightweight session branching using **git worktree**.
- **Tasks:**
 - ❑ Implement **session.branch(name: str) -> SessionBranch** to create an isolated Git worktree in a temporary directory.
 - ❑ Implement **branch.close()** to forcefully remove the worktree and purge all created files.

- ☐ Implement `session.merge(branch)` to merge branch file changes back into the main session.
 - **Acceptance Criteria:** Files created or deleted in `branch` leave zero footprint on `main` after `branch.close()`.
-

Phase 2: Agent Usability & Safety

Goal: Prevent host OS damage and optimize LLM token usage for agent workflows.

◆ Issue 2.1: Safety Guard for Un-rollbackable Global Commands

- **Description:** Intercept commands that affect global OS state and cannot be isolated by Git.
- **Tasks:**
 - ☐ Create `SafetyGuard` module to inspect command strings before execution.
 - ☐ Detect global management tools (`sudo`, `systemctl`, `service`, `apt`, `brew`, `yum`).
 - ☐ Raise `GlobalStateError` with an actionable LLM prompt explanation.
- **Acceptance Criteria:** Executing `sandbox.run("sudo apt-get install nginx")` inside a branch raises `GlobalStateError`.

◆ Issue 2.2: LLM Output Sanitization & Token Saver (`CommandResult`)

- **Description:** Format execution results into token-optimized JSON to save API costs.
 - **Tasks:**
 - ☐ Implement `CommandResult.to_llm_json()` method.
 - ☐ Strip ANSI escape color codes from `stdout/stderr`.
 - ☐ Implement smart truncation/summarization for verbose outputs (e.g., `pip install`, `npm install`).
 - **Acceptance Criteria:** Output of 1,000-line build logs is reduced to a concise JSON summary containing exit code, modified files, and error tail.
-

Phase 3: Ecosystem & Docker Integration

Goal: Support multi-container agent testing and enable one-click integration via MCP.

◆ Issue 3.1: Docker Context Isolation

- **Description:** Prevent port and container name collisions during parallel agent branching.
- **Tasks:**
 - ☐ Automatically inject unique `COMPOSE_PROJECT_NAME` per `SessionBranch`.
 - ☐ Track spawned containers and ensure `docker-compose down -v` is executed on `branch.close()`.
- **Acceptance Criteria:** Two parallel branches can run `docker-compose up` without container name or network collisions.

◆ Issue 3.2: Model Context Protocol (MCP) Server (`ramify-mcp`)

- **Description:** Package `ramify` as a standalone MCP server to be used directly by Claude Desktop, Cursor, and Claude Code.

- **Tasks:**
 - ☐ Implement MCP tools: `ramify_run`, `ramify_branch`, `ramify_close`, `ramify_merge`.
 - ☐ Publish `ramify-mcp` package to PyPI.
 - **Acceptance Criteria:** Users can attach `ramify` to Claude Desktop via `claude_desktop_config.json`.
-

How to use this document with GitHub Issues / Claude Code

To create GitHub Issues from this roadmap using Claude Code:

```
# Example prompt for Claude Code:  
"Read ROADMAP.md and create a GitHub Issue for 'Issue 1.1: Core Stateful  
Session' using the gh CLI."
```